Approach 2: Convex Hull + Fuzzy Boundary Confidence

Predicting Location When No Visit Data Is Present

> IMPORTANT
>
> This document is the authoritative design spec for **Approach 2** of the PS-3 pipeline. Approach 1 (Bayesian grid fusion) runs when `len(visits_data) > 0`. This approach runs in the `else` branch — the **cold-start** scenario — using only parsed address text clues: localities and landmarks pulled from `localities.csv` and `landmarks_poi.csv`.

The Problem Being Solved

When `inference.py` receives an address with **zero field visits**, the current code path still runs the full Bayesian grid — but the grid is just the raw geocoder prior (a single Gaussian centred at the geocoder point). It has no new evidence to update on, so the grid does nothing useful. The confidence radius it returns is purely a function of geocoder precision — coarse, static, and uninformative.

What we actually have in the no-visit case:

The address text (e.g., `"6th Cross, 5th Main, ಚರ್ಚ್ ಹತ್ತಿರ, Kuvempu Layt, Kaveripura – 960102"`) gives the upstream parser several discrete spatial clues:

- One or more **Localities** (joinable to `localities.csv` → `centroid_x`, `centroid_y`)
- One or more **Landmarks / POIs** (joinable to `landmarks_poi.csv` → `x`, `y`)
- The **Geocoder Point** itself (from `baseline_geocodes.csv`)

These clues are **not** probability distributions — they are known reference points in the coordinate system. The question is: *given these scattered pins, where is the actual address, and how confident should we be?*

The proposed answer: **build a Convex Hull around the pins, find its centroid as the prediction, and use a Normal Distribution at each hull edge to derive a principled confidence radius.** This document explains exactly why and how.

Section 1: The Mental Picture (No Math Yet)

Imagine you have a map of Kaveripura. You drop three coloured pins:

- 📍 **Red —** The geocoder's best guess (`geocoder_x` , `geocoder_y`)
- 📍 **Blue —** Centroid of "Kuvempu Layout" locality (`T1-L03` : `x=1112.8, y=-196.7`)
- 📍 **Green —** Nearest landmark, e.g., a Ganesh Temple (`LM0004` : `x=-3155.2, y=823.8`)

Now you stretch a rubber band around the outermost pins. That shape is the **Convex Hull**.

Your best guess for where the house is? The geometric centre of that rubber band — the **centroid of the hull**.

But here is the key insight: each pin is itself fuzzy. A locality centroid doesn't mean the house is exactly at that centroid — it means the house could be *anywhere inside* that locality. The rubber band boundary is therefore not a sharp wall; it should have a **gradual fade —** high probability inside, declining probability as you move outward.

That fade is exactly what a **Normal Distribution (bell curve) placed on each edge** gives you. The boundary is the mean ($\mu = 0$μ = 0 distance from the edge), and the spread (σσ) tells you how much the boundary should "bleed" based on how imprecise the two pins forming that edge are.

Section 2: The Math, Step By Step

Step 1 — Collect Spatial Evidence Points

For each address, gather all available spatial clues into a list of **Evidence Points**, each carrying a coordinate and an initial uncertainty σσ:

Evidence Type	Source Column	Default $\sigma\sigma$ (metres)	Why this value
Geocoder (rooftop)	`baseline_geocodes.csv`	**25 m**	Matches `SpatialPrior.get_precision_sigma('rooftop')` already in your code
Geocoder (street)	`baseline_geocodes.csv`	**110 m**	`get_precision_sigma('street')`
Geocoder (locality)	`baseline_geocodes.csv`	**390 m**	`get_precision_sigma('locality')`
Geocoder (pincode)	`baseline_geocodes.csv`	**1400 m**	`get_precision_sigma('pincode')`
Locality centroid	`localities.csv` → `centroid_x`, `centroid_y`	**350 m**	A locality in your data spans ~700 m; centroid is at the middle
Landmark / POI	`landmarks_poi.csv` → `x`, `y`	**50 m**	POIs are point features, small inherent error

NOTE

These $\sigma\sigma$ values are directly consistent with the `SpatialPrior` class already in your codebase — they are not arbitrary.

Step 2 — Compute the Convex Hull

With nn evidence points $\{P_1, P_2, \ldots, P_n\}${P1, P2, ..., Pn} (each a 2D coordinate), compute the **Convex Hull** — the smallest convex polygon that encloses all points.

```
python
from scipy.spatial import ConvexHull
import numpy as np

points = np.array([[x1, y1], [x2, y2], ...])  # one row per evidence point
hull = ConvexHull(points)
# hull.vertices → indices of the points forming the hull boundary
# hull.simplices → pairs of indices forming each hull edge
```

Edge case handling (important for your real data):

Number of Evidence Points	What Happens	Fallback
1 point	Hull is undefined (a single point has no edges)	Use that point as prediction; use its σσ directly as the radius
2 points	Hull collapses to a line segment (degenerate)	Predict the midpoint; use the σσ-weighted radius of the line
3+ points	Full convex hull works normally	Run the full algorithm below

Step 3 — Find the Predicted Location (the Centroid)

The **centroid of the hull** is your predicted address location.

For a convex polygon with kk hull vertices $V_1, V_2, \ldots, V_k$ V1, V2, ... , Vk, the centroid is:

$$\hat{x} = \frac{1}{k}\sum_{i=1}^{k} V_{i,x}, \quad \hat{y} = \frac{1}{k}\sum_{i=1}^{k} V_{i,y}$$

x^ = k1 i=1∑k Vi,x, y^ = k1 i=1∑k Vi,y

This is a simple arithmetic mean of the hull vertex coordinates (not all evidence points, just the ones on the boundary).

> TIP
>
> **Why the hull centroid, not the mean of all points?** The mean of all evidence points is biased toward whichever type of evidence you have the most of. If you look up 10 landmarks and 1 locality, the mean drifts toward the landmark cluster even if the locality is more authoritative. The hull centroid is purely geometric — it balances the *spatial extent* of your knowledge.

Step 4 — Assign Uncertainty to Each Hull Edge

Each edge of the hull connects two of the original evidence points. Let's call them AA and BB, with uncertainties σ_A σA and σ_B σB.

The uncertainty of the edge is the **inverse-variance weighted average** of the two point uncertainties:

$$\sigma_{\text{edge}} = \frac{\sigma_A^{-2} \cdot \sigma_A + \sigma_B^{-2} \cdot \sigma_B}{\sigma_A^{-2} + \sigma_B^{-2}} = \frac{\frac{1}{\sigma_A} + \frac{1}{\sigma_B}}{\frac{1}{\sigma_A^2} + \frac{1}{\sigma_B^2}}$$

σedge = σA−2 + σB−2σA−2 ⋅ σA + σB−2 ⋅ σB = σA21 + σB21σA1 + σB1

Or in the simpler, close-enough form (when $\sigma_A \approx \sigma_B$ σA ≈ σB):

$$\sigma_{\text{edge}} \approx \frac{\sigma_A + \sigma_B}{2}$$

σedge ≈ 2σA + σB

In plain English: an edge between two imprecise locality centroids gets a wide σσ (e.g., 370 m). An edge between two precise POI landmarks gets a narrow σσ (e.g., 50 m). This directly controls how much the confidence boundary "bleeds" outward at that edge.

Step 5 — Form a Normal Distribution at Every Edge

This is the core of the suggestion you received.

For each hull edge, picture yourself standing on that edge and looking outward (away from the hull's interior). Define a **1D coordinate** dd that measures distance from the edge in the outward direction ($d = 0$d = 0 at the edge, $d > 0$d > 0 outside, $d < 0$d < 0 inside).

Now place a Normal Distribution on this axis:

$$f(d) = N(d \mid \mu = 0, \sigma^2 = \sigma^2_{\text{edge}})$$

f (d) = N (d ∣ μ = 0, σ2 = σedge2)

This distribution is centred at the edge ($\mu = 0$μ = 0), meaning the edge is the most likely boundary location. The σ_{edge} σedge controls how much probability mass leaks outside.

What this represents geometrically: instead of a sharp "inside/outside" boundary, you have a soft boundary. A house 50 m outside the hull edge formed by two locality centroids ($\sigma_{edge} \approx 370$ σedge ≈ 370 m) is still quite plausible. A house 50 m outside a hull edge formed by two POIs ($\sigma_{edge} \approx 50$σedge ≈ 50 m) is already at the 1-sigma tail — much less likely.

Step 6 — Derive the Confidence Radius at Each Edge

You want to find the distance D_{edge} Dedge such that, at your chosen confidence level αα (e.g., 80% or 90%), the probability of the true location being beyond D_{edge} Dedge from this edge is at most $(1 - \alpha)$(1 − α).

For a one-sided Normal Distribution:

$$D_{\text{edge}}(\alpha) = z_\alpha \cdot \sigma_{\text{edge}}$$

Dedge(α) = zα ⋅ σedge

where z_α zα is the standard Normal z-score for the one-sided coverage level:

Target Confidence αα	z_α zα	What it means
80%	0.842	Expand each edge by $0.842 \times \sigma_{\text{edge}}$ 0.842 × σedge
90%	1.282	Expand each edge by $1.282 \times \sigma_{\text{edge}}$ 1.282 × σedge
95%	1.645	Expand each edge by $1.645 \times \sigma_{\text{edge}}$ 1.645 × σedge

NOTE

We use the **one-sided** z-score, not the two-sided one. Because we only care about the probability of being **outside** the hull — the "inside" is already covered by construction. A one-sided 80% interval uses $z = 0.842$z = 0.842 (not $z = 1.28$z = 1.28 which is two-sided 80%).

Pushing the edge outward: Each edge is a line segment in 2D. To expand it by D_{edge} Dedge, translate it outward along its **outward unit normal vector** $\hat{n}_{\text{edge}}$ n^edge (a vector perpendicular to the edge, pointing away from the hull centroid):

$$\text{Expanded Edge} = \text{Original Edge} + D_{\text{edge}} \cdot \hat{n}_{\text{edge}}$$

Expanded Edge = Original Edge + Dedge ⋅ n^edge

Do this for every edge of the hull. You now have an expanded polygon — call it the **Confidence Hull**.

Step 7 — Compute the Final Confidence Radius

The confidence radius is simply the maximum distance from the predicted location (hull centroid $\hat{P}$ P^) to any vertex of the **Confidence Hull**:

$$R_\alpha = \max_{V \in \text{ Confidence Hull vertices}} \|V - \hat{P}\|$$

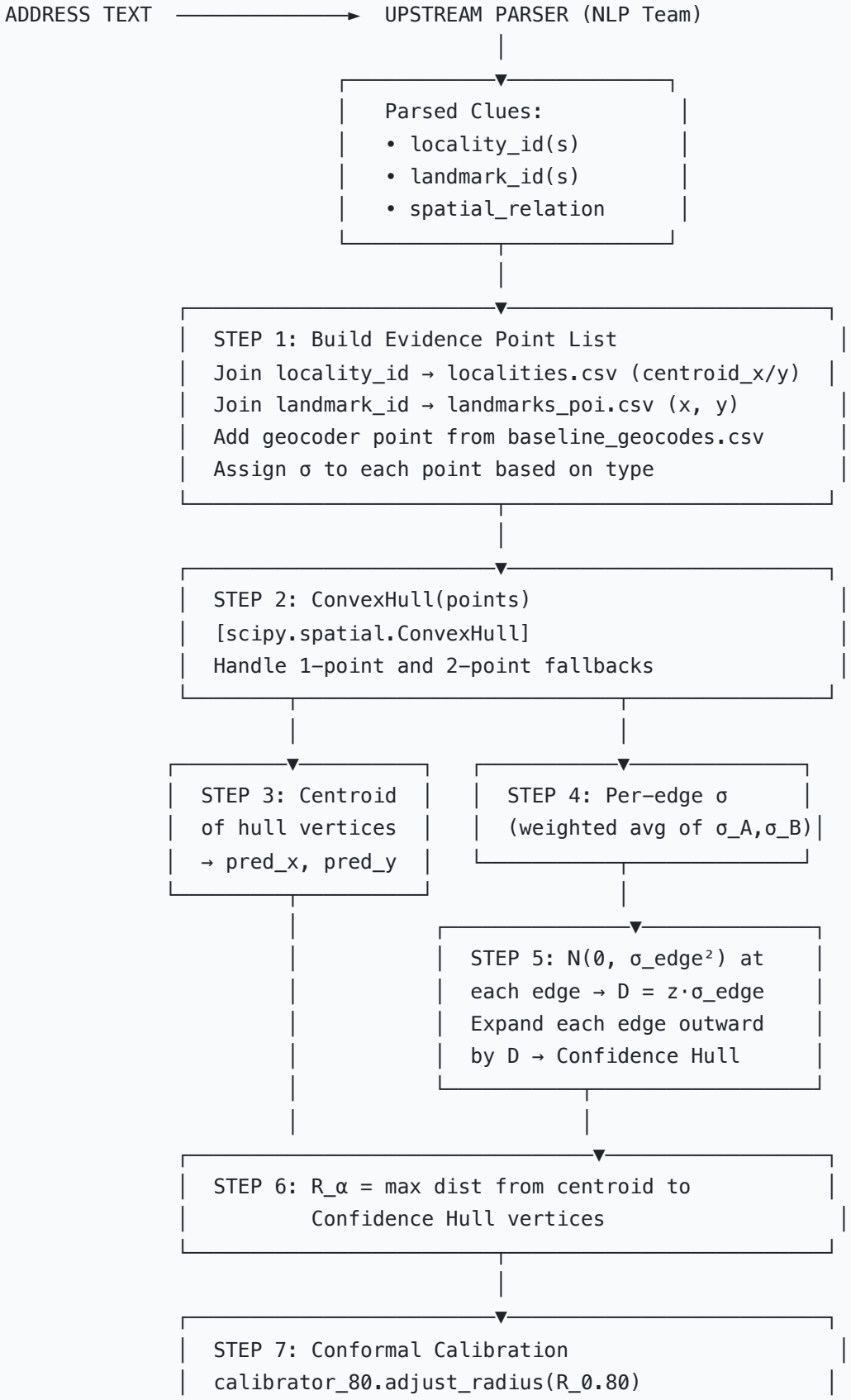
Rα = V ∈Confidence Hull verticesmax ∥V − P^∥

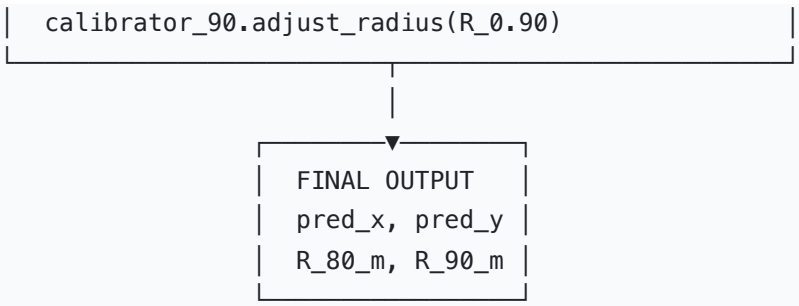
This scalar R_α Rα is your final output — plug it directly into the same output dictionary format as Approach 1:

```
python
{
    "predicted_x":        pred_x,
    "predicted_y":        pred_y,
    "raw_radius_80_m":    R_0.80,
    "calibrated_radius_80_m": calibrator_80.adjust_radius(R_0.80),
    "raw_radius_90_m":    R_0.90,
    "calibrated_radius_90_m": calibrator_90.adjust_radius(R_0.90),
    "method": "Approach 2 — Convex Hull + Fuzzy Boundary (No Visits)"
}
```

> TIP
>
> **Reuse your existing `ConformalCalibrator`** (`conformal.py`) to calibrate Approach 2's raw radii, exactly the same way Approach 1 does. The conformal calibration is evidence-source-agnostic — it just scales the radius to match empirical error on held-out data.

Section 3: Complete Workflow Diagram





Section 4: Where This Slots Into `inference.py`

Currently `inference.py` has this structure:

```
python
if len(visits_data) > 0:
    # Approach 1: Bayesian Grid Fusion ...
else:
    enriched_visits = pd.DataFrame()
    num_useful = 0

# Then always runs BayesianFuser.fuse(...)
```

You need to **split the else branch** into Approach 2:

```
python
if len(visits_data) > 0:
    # Approach 1: existing Bayesian Grid logic (unchanged)
    ...
    pred_x, pred_y, raw_r80, raw_r90 = approach_1_output
else:
    # Approach 2: Convex Hull + Fuzzy Boundary
    from src.convex_hull_predictor import ConvexHullPredictor
    pred_x, pred_y, raw_r80, raw_r90 = ConvexHullPredictor.predict(
        geocoder_x, geocoder_y, precision,
        address_features,   # contains struct_locality_id, struct_landmark_type
        localities_df,      # loaded from localities.csv
        landmarks_df,       # loaded from landmarks_poi.csv
        alpha_80=0.80,
        alpha_90=0.90
    )
    method_desc = "Approach 2 — Convex Hull + Fuzzy Boundary (No Visits)"
```

The new module to create is `src/convex_hull_predictor.py`.

Section 5: Is This Better Than the Alternatives?

vs. Current No-Visit Behaviour (Bayesian Prior Only)

Metric	Current Behaviour	Approach 2
Predicted point	Geocoder centroid	Hull centroid of geocoder + localities + landmarks
Radius source	Purely geocoder precision σ (e.g., 390 m for locality)	Adaptive: shrinks when clues are close, expands when they're spread
Uses parsed text?	❌ No	✅ Yes
Honest about spatial spread?	❌ No (single Gaussian, ignores clue geometry)	✅ Yes (hull captures the true spread of evidence)
Verdict	Treats all locality-precision addresses identically	Differentiates based on specific clue geometry

Approach 2 is unambiguously better than the current no-visit fallback, because it actually uses the parsed text data that the upstream NLP team is producing for you.

vs. Simple Mean/Centroid of All Points

You could just average all evidence points and use the max-distance-to-point as radius. Approach 2 is better because:

- The Convex Hull centroid is **not biased by clue count** (having 5 landmarks near one spot doesn't unfairly pull the mean there)
- The **per-edge σ** gives honest, non-uniform confidence boundaries — not a circle, but a polygon that is tight where your clues are precise and loose where they are imprecise

vs. Approach 1 (Bayesian Grid) on No-Visit Addresses

	Approach 1 (grid) on no-visit	Approach 2 (convex hull)
Computation	Heavy: full N×N grid computed	Light: O(n log n) hull computation
Evidence used	Only the geocoder prior (grid has no visits to update on)	Geocoder + localities + landmarks (more evidence actually used)
Confidence region shape	Circle (from `compute_radius_from_grid`)	Irregular polygon (matches real evidence geometry)
When visits arrive later	N/A	Approach 1 takes over cleanly
Verdict	Overkill and wastes the text clues	Right tool for the job in the cold-start case

> IMPORTANT
>
> Approach 2 is not "worse" than Approach 1 in the no-visit case — it is **more appropriate**. Approach 1 needs GPS point clouds to shine. Approach 2 is purpose-built for text-only clues.

Honest Limitations

> WARNING
>
> **This approach inherits the quality of the upstream parser.** If the parser gives you the wrong locality or a landmark that is 500 m away from the address, the hull centroid will be wrong. Approach 2 is only as good as the parsed clue coordinates. Conformal calibration (`ConformalCalibrator`) will widen the radius to compensate for systematic parser errors if calibrated on enough held-out examples.

> WARNING
>
> **With very few evidence points (1 or 2), the "hull" degenerates.** The 1-point and 2-point fallbacks described in Step 2 are not just edge cases — they will be common for addresses where the parser finds nothing except the geocoder point. Implement those fallbacks carefully.

Section 6: Summary in One Paragraph

Approach 2 takes all available spatial clues for an address (geocoder point, resolved locality centroids from `localities.csv`, POI coordinates from `landmarks_poi.csv`), wraps a convex hull around them, and declares the hull's geometric centroid as the predicted location. It then asks: *"how blurry is each boundary of this hull?"* by averaging the precisions of the two endpoint clues that form each edge (σ_{edge} σedge). A Normal Distribution centred on each edge converts this blurriness into a concrete outward expansion distance — $z_\alpha \cdot \sigma_{\text{edge}}$ zα · σedge — for a chosen confidence level αα. Expanding all edges outward by their respective distances produces a larger polygon (the Confidence Hull), and the maximum distance from the centroid to that polygon's vertices is the final confidence radius. This radius is then passed through the existing `ConformalCalibrator` for empirical correction, and the output is formatted identically to Approach 1, so the rest of the pipeline sees no difference.